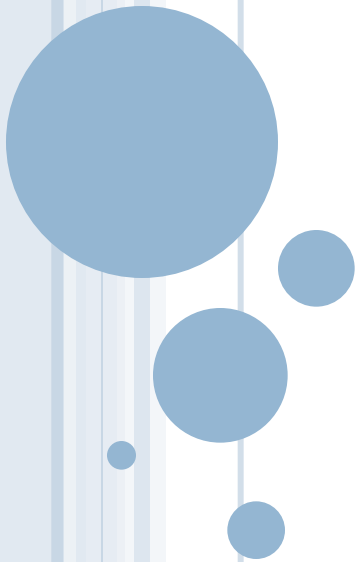


CHAPTER - 1

ON

INTRODUCTION AND OVERVIEW OF DATA STRUCTURES



OBJECTIVES

Know the basics of Data Structures

Types of data structures

Main Operations on Data Structures



DATA TYPE AND DATA STRUCTURE

Data type

- Values for variables
- Operations on those values

Ex : int, float, char ... etc.

Data Structure

The logical and mathematical model of a particular organization of data is called a data structure.

A **data structure** is a particular way or an arrangement of storing and organizing data in a computer's memory or disk storage so that it can be used efficiently.



AN EXAMPLE OF SEVERAL COMMON DATA STRUCTURES:

- Arrays
- Stacks
- Queues
- Trees
- Graph
- Linked lists



DATA STRUCTURE OPERATIONS

The following four operations play a major role:

Traversing

Accessing each record exactly once so that certain items in the record may be processed. (This accessing or processing is sometimes called 'visiting' the records.)

Searching

Finding the location of the record with a given key value, or finding the locations of all records, which satisfy one or more conditions.

Inserting

Adding new records to the structure.

Deleting

Removing a record from the structure.



DATA STRUCTURE OPERATIONS (CONTINUED)

The following two operations, which are used in special situations, will also be considered:

Sorting:

Arranging the records in some logical order

Merging:

Combining the records in two different sorted files into a single sorted files



Types of Data Structure

There are two types of data structure

Linear Data Structure

A data structure is said to be linear if its elements form a sequence, or in other words a linear list.

- Array
- Stack
- Queue
- Linked List

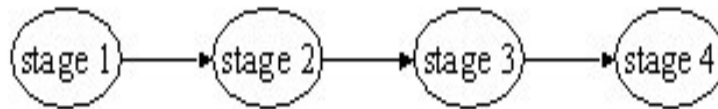
Non-Linear Data Structure

A non-linear structure is mainly used to represent data containing a hierarchical relationship between elements.

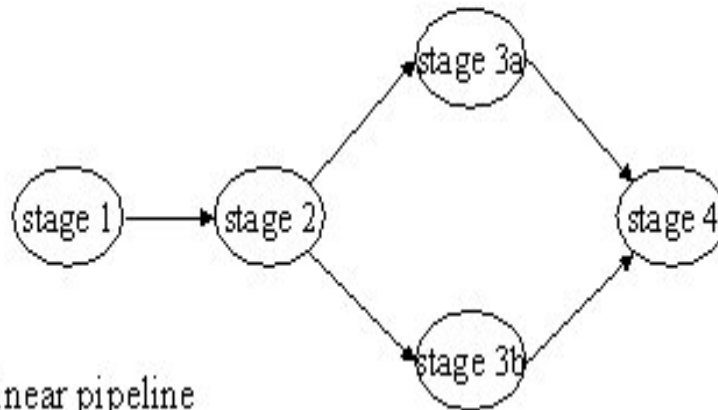
- Tree
- Graph



DIFFERENCE BETWEEN LINEAR AND NON LINEAR DATA STRUCTURES



linear pipeline



non-linear pipeline



ALGORITHMS

An Algorithm is a finite step – by – step list of well defined instructions for solving a particular problem. It is used to manipulate the data contained in the data structures as in searching and sorting. It states explicitly how the data will be manipulated.



COMPLEXITY

The complexity of an algorithm is a function describing the efficiency of the algorithm in terms of the amount of data the algorithm must process. There are two main complexity measures of the efficiency of an algorithm:

Time complexity is a function describing the amount of time an algorithm takes in terms of the amount of input to the algorithm.

Space complexity is a function describing the amount of memory (space) an algorithm takes in terms of the amount of input to the algorithm.



Discussion

- Once data is organized into forms such as trees, stacks and queues, standard algorithms can be used to process them.
- Properly organized data lead to easy-to understand.



Recursion and Iteration

$$\text{Factorial } (n) = \begin{cases} 1 & \text{if } n = 0 \\ n \times (n-1) \times (n-2) \times \dots \times 3 \times 2 \times 1 & \text{if } n > 0 \end{cases}$$

Recursion: A repetitive algorithm uses recursion whenever the algorithm appears within the definition itself.

Iteration : A repetitive algorithm is defined iteratively whenever the definition involves only the algorithm parameter(s) and not the algorithm itself. (Repetition of some statements using loops)



Iterative Solution

Let's write an algorithm to solve the factorial problem iteratively. This solution usually involves using a loop such as the one shown in Algorithm 2-1.

2-1 Iterative Factorial Algorithm

```
Algorithm iterativeFactorial (n)
Calculates the factorial of a number using a loop.
  Pre  n  is the number to be raised factorially
  Post n! is returned
1 set i to 1
2 set factN to 1
3 loop (i <= n)
  1 set factN to factN * i
  2 increment i
4 end loop
5 return factN
end iterativeFactorial
```



Recursive Solution

Now let's write the same algorithm recursively. The recursive solution does not need a loop; recursion is itself repetition. In the recursive version, we let the factorial algorithm call itself, each time with a different set of parameters. The algorithm for recursive factorial is shown in Algorithm 2-2.

2-2 Recursive Factorial

```
Algorithm recursiveFactorial (n)
Calculates factorial of a number using recursion.
  Pre    n is the number being raised factorially
  Post   n! is returned
1 if (n equals 0)
  1 return 1
2 else
  1 return (n * recursiveFactorial (n - 1))
3 end if
end recursiveFactorial
```



Pointer

- Pointer is a variable contain the address of another variable.
- All operation perform on pointers are done through two operators '*' and '&'. '&' is a unary operator that returns a memory address of a variable. '*' return value stored at a memory location stored in a pointer.

General form of pointer declaration is -
datatype* name;
int *p

